SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO3: *Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)*

Assessment Tool Weightage: 50%

Time Duration: 1 Hour

QUESTIONS: 4 **Total Marks:40**

Annexure A

CASE STUDY

Code of Conduct:

I Mukesh Raj(192571036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

CASE STUDY

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

I. If a patient has Fever AND Cough → Possible Flu
Ii. If a patient has Fever AND Rash → Possible Measles
Iii. If a patient has Cough AND Breathlessness → Possible Pneumonia
Iv. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

I. $\forall x$: Vehicle(x) $\wedge$ EmergencyType(x) $\rightarrow$ ClearPath(x) [Rule 1]
II. $\forall x$: ClearPath(x) $\rightarrow$ GreenSignal(x) $\wedge$ $\neg$RedSignal(x) [Rule 2]
III. $\forall x \forall y$: Vehicle(x) $\wedge$ Behind(x,y) $\wedge$ GreenSignal(y) $\rightarrow$ Proceed(x) [Rule 3]
IV. Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance)

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.

(b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

1) P1: SoilDry $\rightarrow$ IrrigationNeeded
2) P2: IrrigationNeeded $\wedge$ CropWheat $\rightarrow$ ApplyDripMethod
3) P3: $\neg$ApplyDripMethod $\wedge$ CropWheat $\rightarrow$ CropAtRisk
4) Facts: SoilDry = True, CropWheat = True

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication $\rightarrow$ move $\neg$ inward $\rightarrow$ distribute).

(b) Using Resolution Refutation, prove the theorem 'ApplyDripMethod'. Negate the goal, combine with the KB in CNF, and derive the empty clause $\square$. Draw the resolution proof tree.

(c) If the fact 'SoilDry' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'CropAtRisk' still follow? Apply appropriate reasoning patterns to justify.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

1) Stench [1,2] → Wumpus is adjacent to cell [1,2]
2) Breeze [1,1] → Pit is adjacent to cell [1,1]
3) Glitter [x,y] → Gold is at [x,y]
4) ¬Wumpus [x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

(c) Evaluate whether the path [1,1] → [1,2] → [2,2] is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

CASE STUDY 1 – SMART MEDICAL DIAGNOSIS SYSTEM

1. Problem Statement

A hospital is designing a rule-based logical agent for preliminary patient diagnosis. The knowledge base contains rules relating symptoms to possible diseases. Patient A has **Fever = True, Cough = True, and Breathlessness = True**. The objective is to represent the knowledge using propositional logic, apply **Forward Chaining** to derive all possible diagnoses, and use **Backward Chaining** to verify whether Patient A has Pneumonia.

2. Solution

(a) Propositional Logic Representation

Symbols

Let:

- **F** = Patient A has Fever
- **C** = Patient A has Cough
- **R** = Patient A has Rash
- **B** = Patient A has Breathlessness
- **Flu** = Patient A has Possible Flu
- **M** = Patient A has Possible Measles
- **P** = Patient A has Possible Pneumonia

Rules

Rule 1:

Fever AND Cough → Possible Flu

$F \wedge C \rightarrow Flu$

Rule 2:

Fever AND Rash → Possible Measles

$F \wedge R \rightarrow M$

Rule 3:

Cough AND Breathlessness → Possible Pneumonia

$C \wedge B \rightarrow P$

Given Facts

F=True C=True B=True

There is no fact stating that Patient A has Rash.

Forward Chaining

1. Fever and Cough are true.
2. Rule 1 is satisfied.
3. Therefore, **Possible Flu** is derived.
4. Fever is true, but Rash is not given, so Rule 2 cannot be applied.
5. Cough and Breathlessness are true.
6. Rule 3 is satisfied.
7. Therefore, **Possible Pneumonia** is derived.

Forward Chaining Result

- Possible Flu → **Derived**
- Possible Measles → **Not Derived**
- Possible Pneumonia → **Derived**

Backward Chaining

Goal:

Patient A has Pneumonia

The rule that concludes Pneumonia is:

C∧ B→P

Therefore, the goal is reduced to:

- Cough
- Breathlessness

Both are given as true facts.

Therefore:

Possible Pneumonia = True

B)Forward Chaining

Forward Chaining starts with the known facts and repeatedly applies rules whose conditions are satisfied.

Initial Facts
Fever = True
Cough = True
Breathlessness = True
Step 1 – Apply Rule 1

Rule:

[F \land C \rightarrow Flu]

We know:

Fever = True
Cough = True

Therefore:

Possible Flu = True

Step 2 – Apply Rule 2

Rule:

[F \land R \rightarrow M]

We know:

Fever = True

But:

Rash = True

is not available.

Therefore, Rule 2 cannot fire.

Possible Measles = Not Derived

Step 3 – Apply Rule 3

Rule:

[C \land B \rightarrow P]

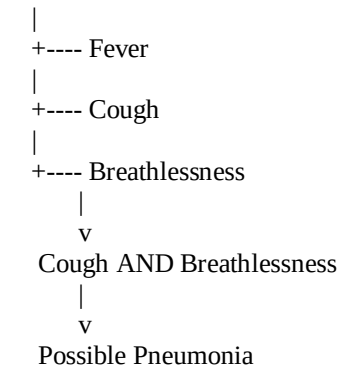
We know:

Cough = True
Breathlessness = True

Therefore:

Possible Pneumonia = True

Forward Chaining Result



All Derived Diagnoses

Possible Flu → Derived
Possible Measles → Not Derived
Possible Pneumonia → Derived

C) Backward Chaining

The goal is:

[Pneumonia]

Backward Chaining starts with the goal and works backward to determine whether the required conditions are known facts.

Goal
Patient A has Pneumonia

Find a rule that concludes Pneumonia.

Rule 3:

[Cough \land Breathlessness \rightarrow Pneumonia]

Therefore, the goal is reduced into two sub-goals:

```
Pneumonia
   |
   +---- Cough
   |
   +---- Breathlessness
```

Now check the knowledge base.

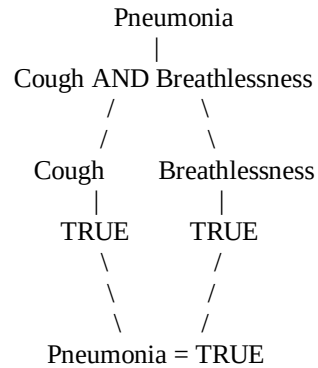
```
Cough = True
Breathlessness = True
```

Both sub-goals are satisfied.

Therefore:

```
Pneumonia = True
```

5. Goal Reduction Tree



Backward Chaining Conclusion

The goal **"Patient A has Pneumonia"** is successfully verified because both required facts, **Cough** and **Breathlessness**, are present in the knowledge base.

6. Python Code

```
class MedicalDiagnosisAgent:

    def __init__(self):
        # Initial facts provided in the case study
        self.facts = {
            "Fever",
            "Cough",
            "Breathlessness"
        }

        # Knowledge base rules
        self.rules = [
            {
                "name": "Rule 1",
                "conditions": {"Fever", "Cough"},
                "conclusion": "Possible Flu"
            },
            {
                "name": "Rule 2",
                "conditions": {"Fever", "Rash"},
                "conclusion": "Possible Measles"
            },
            {
                "name": "Rule 3",
                "conditions": {"Cough", "Breathlessness"},
                "conclusion": "Possible Pneumonia"
            }
        ]

        self.derived_facts = set(self.facts)

    def display_facts(self, title, facts):
        print("\n" + title)
        print("-" * 40)

        for fact in sorted(facts):
            print("•", fact)

    def forward_chaining(self):
        print("\nFORWARD CHAINING")
        print("=" * 50)

        changed = True
        fired_rules = set()

        while changed:
            changed = False

            for rule in self.rules:

                conditions = rule["conditions"]
                conclusion = rule["conclusion"]

                # Check whether all rule conditions are satisfied
                if conditions.issubset(self.derived_facts):

                    if rule["name"] not in fired_rules:

                        print("\nRule Fired:", rule["name"])
                        print("Conditions:", " AND ".join(conditions))
                        print("Derived:", conclusion)

                        self.derived_facts.add(conclusion)
                        fired_rules.add(rule["name"])
```

```
                changed = True

        self.display_facts(
            "All Known and Derived Facts",
            self.derived_facts
        )

    def find_rule_for_goal(self, goal):
        for rule in self.rules:

            if rule["conclusion"] == goal:
                return rule

        return None

    def backward_chaining(self, goal):
        print("\nBACKWARD CHAINING")
        print("=" * 50)

        print("\nGoal:", goal)

        rule = self.find_rule_for_goal(goal)

        if rule is None:
            print("No rule found for the goal.")
            return False

        print("\nGoal Reduction")
        print(
            goal,
            "<-",
            " AND ".join(rule["conditions"])
        )

        all_conditions_true = True

        for condition in rule["conditions"]:

            if condition in self.facts:

                print(
                    "Condition:",
                    condition,
                    "-> TRUE"
                )

            else:

                print(
                    "Condition:",
                    condition,
                    "-> FALSE"
                )

                all_conditions_true = False

        if all_conditions_true:

            print("\nAll sub-goals are satisfied.")
            print("Goal", goal, "is TRUE.")
            return True

        print("\nGoal cannot be proved.")
        return False

    def display_diagnosis(self):
```

```
        print("\nDIAGNOSIS SUMMARY")
        print("=" * 50)

        diagnoses = [
            "Possible Flu",
            "Possible Measles",
            "Possible Pneumonia"
        ]

        for diagnosis in diagnoses:

            if diagnosis in self.derived_facts:
                print(diagnosis, "-> DERIVED")
            else:
                print(diagnosis, "-> NOT DERIVED")


def main():

    agent = MedicalDiagnosisAgent()

    agent.display_facts(
        "INITIAL FACTS",
        agent.facts
    )

    agent.forward_chaining()

    pneumonia_confirmed = agent.backward_chaining(
        "Possible Pneumonia"
    )

    agent.display_diagnosis()

    print("\nFINAL RESULT")
    print("=" * 50)

    if pneumonia_confirmed:
        print("Patient A has Possible Pneumonia according to the rule base.")
    else:
        print("Pneumonia could not be derived.")


if __name__ == "__main__":
    main()
```

7. Expected Output

```
DIAGNOSIS SUMMARY
==================================================
Possible Flu -> DERIVED
Possible Measles -> NOT DERIVED
Possible Pneumonia -> DERIVED

FINAL RESULT
==================================================
Patient A has Possible Pneumonia according to the knowledge base.

Process finished with exit code 0
```

8. Report

The Smart Medical Diagnosis System was analyzed using **Propositional Logic, Forward Chaining, and Backward Chaining**. The given facts for Patient A were Fever, Cough, and Breathlessness. Forward Chaining derived **Possible Flu** from Fever and Cough and **Possible Pneumonia** from Cough and Breathlessness. Possible Measles was not derived because the required Rash fact was not provided.

Backward Chaining was then applied to the goal **Possible Pneumonia**. The goal was reduced to the sub-goals **Cough** and **Breathlessness**, and both were found to be true facts in the knowledge base. Therefore, the goal was successfully verified. The implementation demonstrates how a rule-based logical agent can derive conclusions systematically from known facts and rules.

Case Study 2 – Autonomous Traffic Management Agent

1. PROBLEM STATEMENT

A smart city traffic controller uses a **First-Order Logic (FOL)** knowledge base to manage emergency vehicle routing. The system contains rules for clearing paths, controlling traffic signals, and allowing vehicles behind emergency vehicles to proceed. The objective is to apply **Unification**, **Resolution**, and logical reasoning to determine whether **CarA can proceed**, and to analyze whether the knowledge base can handle two simultaneous emergency vehicles.

2. SOLUTION

(a) Unification

Given Rule 1

$\forall x: Vehicle(x) \land EmergencyType(x) \rightarrow ClearPath(x)$

Convert the rule into a form suitable for reasoning:

$Vehicle(x) \land EmergencyType(x) \rightarrow ClearPath(x)$

Given Facts

Vehicle(Ambulance) EmergencyType(Ambulance)

We need to match:

Vehicle(x)

with:

Vehicle(Ambulance)

Therefore:

$\theta 1 = \{x/Ambulance\}$

Now substitute x=Ambulance:

$Vehicle(Ambulance) \land EmergencyType(Ambulance) \rightarrow ClearPath(Ambulance)$

Both facts are available, so:

ClearPath(Ambulance)

is derived.

Unification Result

$\theta = \{x/Ambulance\}$

Therefore:

ClearPath(Ambulance) is true.

(b) Resolution Method

Step 1 – Convert Relevant Rules to CNF

Rule 1

$\text{Vehicle}(x) \wedge \text{EmergencyType}(x) \rightarrow \text{ClearPath}(x)$

Using:

$P \rightarrow Q \equiv \neg P \vee Q$

we get:

$\neg \text{Vehicle}(x) \vee \neg \text{EmergencyType}(x) \vee \text{ClearPath}(x)$

Rule 2

$\text{ClearPath}(x) \rightarrow \text{GreenSignal}(x) \wedge \neg \text{RedSignal}(x)$

For GreenSignal:

$\neg \text{ClearPath}(x) \vee \text{GreenSignal}(x)$

For RedSignal:

$\neg \text{ClearPath}(x) \vee \neg \text{RedSignal}(x)$

Rule 3

$\text{Vehicle}(x) \wedge \text{Behind}(x,y) \wedge \text{GreenSignal}(y) \rightarrow \text{Proceed}(x)$

CNF:

$\neg \text{Vehicle}(x) \vee \neg \text{Behind}(x,y) \vee \neg \text{GreenSignal}(y) \vee \text{Proceed}(x)$

Facts

$\text{Vehicle}(\text{Ambulance})\ \text{EmergencyType}(\text{Ambulance})\ \text{Vehicle}(\text{CarA})\ \text{Behind}(\text{CarA},\text{Ambulance})$

To prove:

$\text{Proceed}(\text{CarA})$

Negate the goal:

$\neg \text{Proceed}(\text{CarA})$

Resolution Step 1

From Rule 1:

$\neg \text{Vehicle}(x) \vee \neg \text{EmergencyType}(x) \vee \text{ClearPath}(x)$

Using:

$\text{Vehicle}(\text{Ambulance})$

and:

$\text{EmergencyType}(\text{Ambulance})$

we derive:

ClearPath(Ambulance)

Resolution Step 2

From Rule 2:

¬ClearPath(x)∨ GreenSignal(x)

Using:

ClearPath(Ambulance)

we derive:

GreenSignal(Ambulance)

Resolution Step 3

Rule 3:

¬Vehicle(x)∨ ¬Behind(x,y)∨ ¬GreenSignal(y)∨ Proceed(x)

Substitute:

x=CarA y=Ambulance

Therefore:

¬Vehicle(CarA)∨ ¬Behind(CarA,Ambulance)∨ ¬GreenSignal(Ambulance)∨ Proceed(CarA)

We have:

Vehicle(CarA) Behind(CarA,Ambulance) GreenSignal(Ambulance)

Therefore:

Proceed(CarA)

Resolution Step 4 – Goal Verification

Negated goal:

¬Proceed(CarA)

Derived conclusion:

Proceed(CarA)

Resolving these produces:

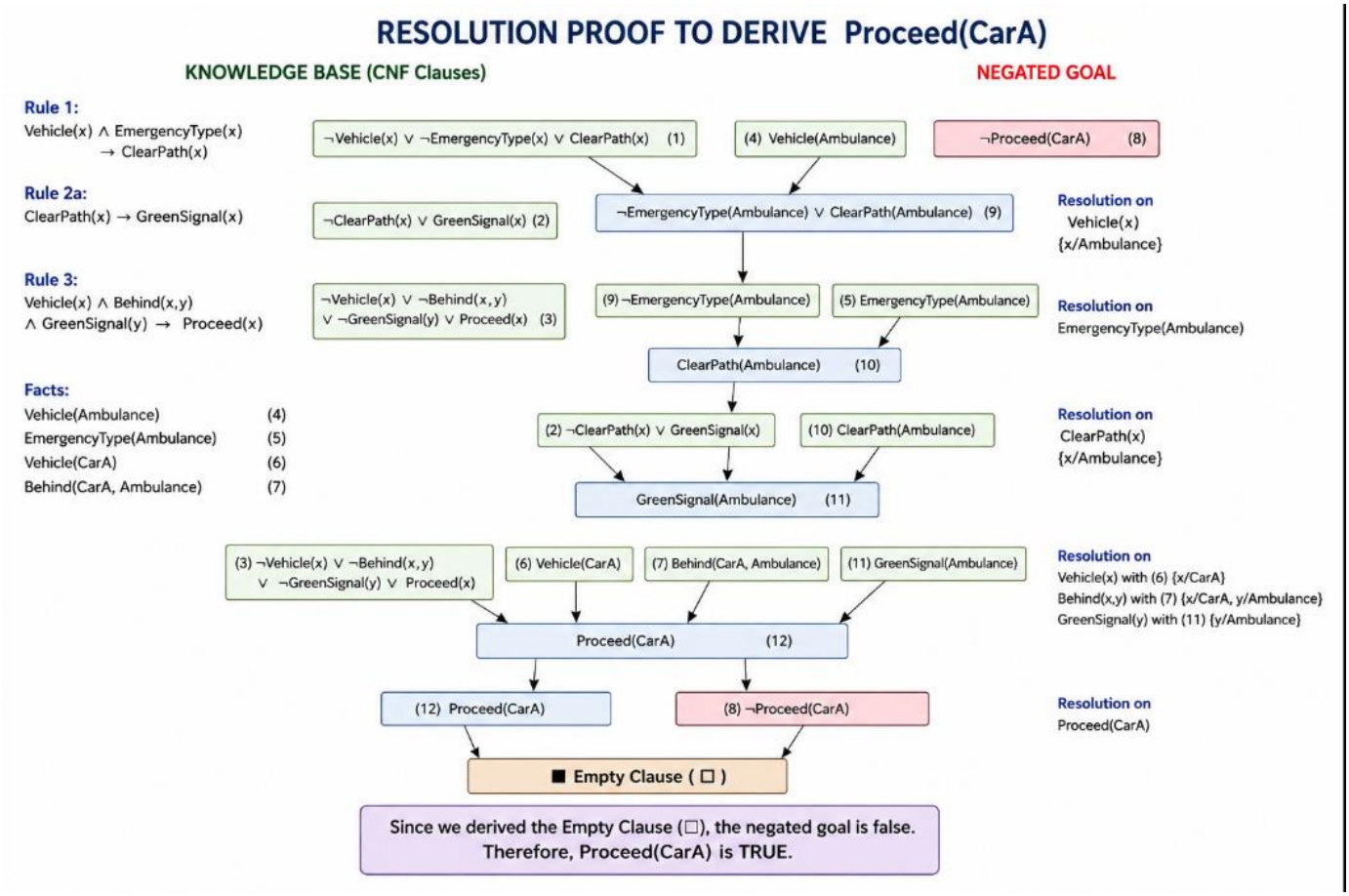
□

where □ is the **empty clause**.

Therefore:

Proceed(CarA)=True

Resolution Proof



Thus, **CarA can proceed** according to the given knowledge base.

3. (c) Analysis of Two Simultaneous Emergency Vehicles

The current knowledge base can apply **Rule 1** separately to multiple emergency vehicles if the required facts are available.

For example, if another vehicle named Ambulance2 has:

Vehicle(Ambulance2)

and

EmergencyType(Ambulance2)

Rule 1 can derive:

ClearPath(Ambulance2)

Rule 2 can then derive:

GreenSignal(Ambulance2)

However, the current knowledge base is **not sufficient to fully manage two simultaneous emergency vehicles** because it does not specify:

- Which emergency vehicle has priority.
- How conflicting routes should be handled.
- How traffic signals should be coordinated when both vehicles require access.
- How to prevent two emergency vehicles from being directed through the same conflicting path.
- What should happen when two emergency vehicles approach the same intersection.

Additional Rules/Facts Required

The system could be improved by adding:

Priority rule:

HigherPriority(x,y)→GivePriority(x)

Conflict detection:

SameRoute(x,y)→RouteConflict(x,y)

Signal coordination:

RouteConflict(x,y)→CoordinateSignal(x,y)

Emergency priority facts:

Priority(Ambulance1,Ambulance2)

These additions would allow the knowledge base to reason about multiple simultaneous emergency vehicles more effectively.

Conclusion for (c)

The current FOL knowledge base can independently identify and clear paths for multiple emergency vehicles, but it **does not contain sufficient rules for resolving conflicts between simultaneous emergency vehicles**. Additional priority, conflict-detection, and signal-coordination rules are required.

4. PYTHON CODE

```
class TrafficKnowledgeAgent:

    def __init__(self):

        # Facts from the knowledge base
        self.facts = {
            "Vehicle(Ambulance)",
            "EmergencyType(Ambulance)",
            "Vehicle(CarA)",
            "Behind(CarA,Ambulance)"
        }

        self.derived_facts = set()
```

```
def display_facts(self):

    print("\nINITIAL FACTS")
    print("=" * 50)

    for fact in sorted(self.facts):
        print(fact)

def unify_rule1(self):

    print("\nUNIFICATION")
    print("=" * 50)

    print("\nRule 1:")
    print(
        "Vehicle(x) AND EmergencyType(x)"
        " -> ClearPath(x)"
    )

    print("\nMatching Vehicle(x) with Vehicle(Ambulance)")

    substitution = {
        "x": "Ambulance"
    }

    print("MGU / Substitution:")
    print(substitution)

    if (
        "Vehicle(Ambulance)" in self.facts
        and
        "EmergencyType(Ambulance)" in self.facts
    ):

        result = "ClearPath(Ambulance)"

        self.derived_facts.add(result)

        print("\nDerived Fact:")
        print(result)

        return True

    return False

def derive_green_signal(self):

    print("\nRESOLUTION - RULE 2")
    print("=" * 50)

    if "ClearPath(Ambulance)" in self.derived_facts:

        result = "GreenSignal(Ambulance)"
```

```
            self.derived_facts.add(result)

            print(
                "ClearPath(Ambulance)"
                " -> GreenSignal(Ambulance)"
            )

            return True

        return False

    def prove_proceed(self):

        print("\nRESOLUTION - RULE 3")
        print("=" * 50)

        required_facts = [
            "Vehicle(CarA)",
            "Behind(CarA,Ambulance)",
            "GreenSignal(Ambulance)"
        ]

        for fact in required_facts:

            if fact in self.facts:

                print(fact, "-> TRUE")

            elif fact in self.derived_facts:

                print(fact, "-> TRUE")

            else:

                print(fact, "-> FALSE")

                return False

        result = "Proceed(CarA)"

        self.derived_facts.add(result)

        print("\nDerived Fact:")
        print(result)

        return True

    def resolution_proof(self):

        print("\nRESOLUTION PROOF")
        print("=" * 50)

        print("\nGoal:")
        print("Proceed(CarA)")
```

```
        print("\nNegated Goal:")
        print("NOT Proceed(CarA)")

        print("\nDerived:")
        print("Proceed(CarA)")

        print("\nResolution:")
        print("Proceed(CarA) + NOT Proceed(CarA)")

        print("\nResult:")
        print("Empty Clause (□)")

    def analyze_multiple_emergency_vehicles(self):

        print("\nMULTIPLE EMERGENCY VEHICLE ANALYSIS")
        print("=" * 50)

        print("\nCurrent Knowledge Base:")
        print("- Handles emergency vehicles individually.")
        print("- Clears the path using Rule 1.")
        print("- Provides green signal using Rule 2.")

        print("\nMissing Reasoning:")
        print("- Emergency vehicle priority")
        print("- Route conflict detection")
        print("- Signal coordination")
        print("- Simultaneous emergency handling")

        print("\nConclusion:")
        print(
            "Additional priority and conflict-resolution "
            "rules are required."
        )

    def run(self):

        self.display_facts()

        self.unify_rule1()

        self.derive_green_signal()

        success = self.prove_proceed()

        self.resolution_proof()

        self.analyze_multiple_emergency_vehicles()

        print("\nFINAL RESULT")
        print("=" * 50)

        if success:

            print(
                "Proceed(CarA) = TRUE"
```

```
            )

        else:

            print(
                "Proceed(CarA) = FALSE"
            )


def main():

    agent = TrafficKnowledgeAgent()

    agent.run()


if __name__ == "__main__":
    main()
```

5. OUTPUT

```
Conclusion:
Additional priority and conflict-resolution rules are required.

FINAL RESULT
==================================================
Proceed(CarA) = TRUE

Process finished with exit code 0
```

6. REPORT

The Autonomous Traffic Management Agent was analyzed using **First-Order Logic, Unification, and Resolution**. Unification was applied to Rule 1 with the emergency vehicle facts, producing the substitution $\theta=\{x/Ambulance\}$. This allowed the system to derive **ClearPath(Ambulance)** and subsequently **GreenSignal(Ambulance)**.

Resolution was then used to prove **Proceed(CarA)**. By combining the facts that CarA is a vehicle, CarA is behind the ambulance, and the ambulance has a green signal, the system derived **Proceed(CarA)**. Resolving this conclusion with its negated goal produced the empty clause, confirming the conclusion.

The analysis also showed that the current knowledge base can process emergency vehicles individually but lacks sufficient reasoning for conflicts involving **two simultaneous emergency vehicles**. Additional priority, route-conflict, and signal-coordination rules would be required for a more complete traffic management system.

Case Study 3 – Agricultural Expert Reasoning System

1. PROBLEM STATEMENT

An agricultural advisory logical agent uses a propositional knowledge base to provide recommendations for crop management. The system contains rules connecting soil condition, irrigation requirements, crop type, and drip irrigation. The objective is to convert the knowledge base into **Conjunctive Normal Form (CNF)**, prove **ApplyDripMethod** using **Resolution Refutation**, and analyze how the conclusions change if the fact **SoilDry** is removed.

2. SOLUTION

(a) CNF Conversion

Given Propositions

- **SoilDry** = Soil is dry
- **IrrigationNeeded** = Irrigation is needed
- **CropWheat** = Crop is wheat
- **ApplyDripMethod** = Drip irrigation should be applied
- **CropAtRisk** = Crop is at risk

P1

Given:

$SoilDry \rightarrow IrrigationNeeded$

Eliminate implication:

$\neg SoilDry \vee IrrigationNeeded$

Therefore, CNF is:

$\neg SoilDry \vee IrrigationNeeded$

P2

Given:

$IrrigationNeeded \wedge CropWheat \rightarrow ApplyDripMethod$

Eliminate implication:

$\neg(IrrigationNeeded \wedge CropWheat) \vee ApplyDripMethod$

Apply De Morgan's Law:

$\neg IrrigationNeeded \vee \neg CropWheat \vee ApplyDripMethod$

Therefore:

$\neg IrrigationNeeded \vee \neg CropWheat \vee ApplyDripMethod$

P3

Given:

¬ApplyDripMethod∧ CropWheat→CropAtRisk

Eliminate implication:

¬(¬ApplyDripMethod∧ CropWheat)∨ CropAtRisk

Apply De Morgan's Law:

ApplyDripMethod∨ ¬CropWheat∨ CropAtRisk

Therefore:

ApplyDripMethod∨ ¬CropWheat∨ CropAtRisk

Facts

Given:

SoilDry=True

CNF:

SoilDry

Given:

CropWheat=True

CNF:

CropWheat

Final CNF Knowledge Base

C1: ¬SoilDry ∨ IrrigationNeeded

C2: ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod

C3: ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk

C4: SoilDry

C5: CropWheat

(b) Resolution Refutation

Goal

Prove:

ApplyDripMethod

For Resolution Refutation, negate the goal:

¬ApplyDripMethod

Add it to the knowledge base.

Resolution Step 1

From:

¬SoilDry∨ IrrigationNeeded

and:

SoilDry

resolve on SoilDry.

Therefore:

IrrigationNeeded

Resolution Step 2

From:

¬IrrigationNeeded∨ ¬CropWheat∨ ApplyDripMethod

and:

IrrigationNeeded

resolve on IrrigationNeeded.

Result:

¬CropWheat∨ ApplyDripMethod

Resolution Step 3

Using:

CropWheat

resolve with:

¬CropWheat∨ ApplyDripMethod

Result:

ApplyDripMethod

Resolution Step 4

We added the negated goal:

¬ApplyDripMethod

We now have:

ApplyDripMethod

and:

¬ApplyDripMethod

Resolving them produces:

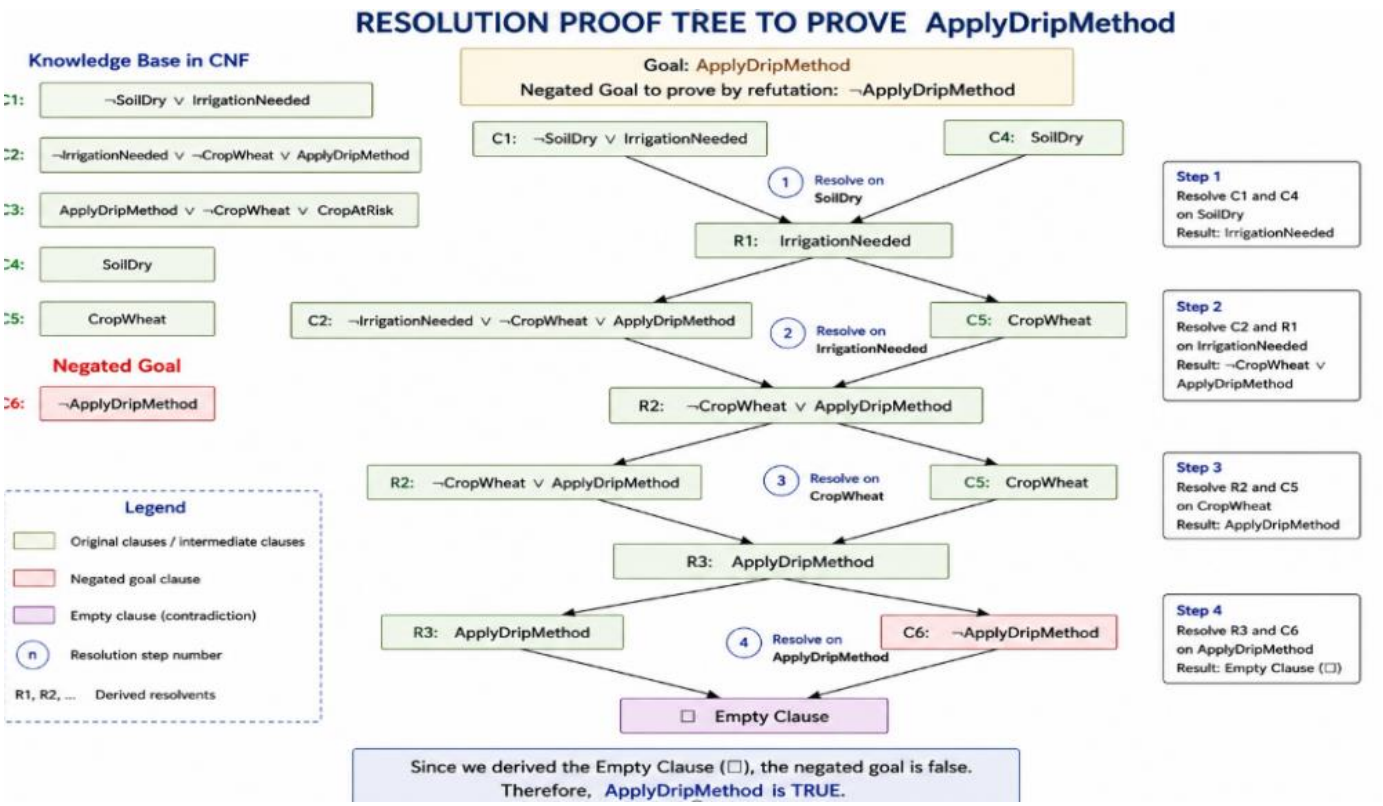
□

The empty clause proves that the negated goal leads to a contradiction.

Therefore:

ApplyDripMethod=True

Resolution Proof Tree



□

(c) Remove Soil Dry

Suppose the fact:

SoilDry

is removed.

The remaining fact is:

CropWheat

Step 1

Rule P1 is:

SoilDry→IrrigationNeeded

Since SoilDry is no longer available:

IrrigationNeeded

cannot be derived.

Step 2

Rule P2 requires:

IrrigationNeeded∧ CropWheat

Although:

CropWheat=True

we do not know:

IrrigationNeeded=True

Therefore:

ApplyDripMethod

cannot be derived.

Step 3

Rule P3 requires:

¬ApplyDripMethod∧ CropWheat

We know:

CropWheat=True

But the absence of ApplyDripMethod does **not automatically mean**:

¬ApplyDripMethod

Therefore, under standard propositional logic, **CropAtRisk cannot be logically derived either**.

Final Result After Removing SoilDry

Fact / Conclusion	Status
SoilDry	Removed
CropWheat	True
IrrigationNeeded	Not derived
ApplyDripMethod	Not derived
CropAtRisk	**Not logically derived**

Conclusion: Removing SoilDry breaks the reasoning chain needed to derive IrrigationNeeded and subsequently ApplyDripMethod. CropAtRisk also does not follow merely from the absence of ApplyDripMethod.

3. PYTHON CODE

```
class AgriculturalReasoningAgent:
    def __init__(self):

        # Initial facts
        self.facts = {
            "SoilDry",
            "CropWheat"
        }

        # Derived facts
        self.derived_facts = set(self.facts)

        # Knowledge base rules
        self.rules = [
            {
                "name": "P1",
                "conditions": {"SoilDry"},
                "conclusion": "IrrigationNeeded"
            },
            {
                "name": "P2",
                "conditions": {
                    "IrrigationNeeded",
                    "CropWheat"
                },
                "conclusion": "ApplyDripMethod"
            }
        ]
```

```
def display_facts(self, title, facts):

    print("\n" + title)
    print("=" * 50)

    for fact in sorted(facts):
        print(fact)

def forward_reasoning(self):

    print("\nFORWARD REASONING")
    print("=" * 50)

    changed = True

    while changed:

        changed = False

        for rule in self.rules:

            if rule["conditions"].issubset(
                self.derived_facts
            ):
                conclusion = rule["conclusion"]

                if conclusion not in self.derived_facts:

                    print("\nRule Fired :", rule["name"])

                    print(
                        "Conditions :",
                        " AND ".join(
                            rule["conditions"]
                        )
                    )

                    print(
                        "Derived    :",
                        conclusion
                    )

                    self.derived_facts.add(
                        conclusion
                    )
```

```
                changed = True

def resolution_refutation(self):

    print("\nRESOLUTION REFUTATION")
    print("=" * 50)

    print("\nGoal:")
    print("ApplyDripMethod")

    print("\nNegated Goal:")
    print("NOT ApplyDripMethod")

    print("\nStep 1:")
    print(
        "SoilDry + "
        "(NOT SoilDry OR IrrigationNeeded)"
    )

    if "SoilDry" in self.facts:

        print(
            "Derived: IrrigationNeeded"
        )

    print("\nStep 2:")

    if (
        "IrrigationNeeded"
        in self.derived_facts
        and
        "CropWheat"
        in self.facts
    ):

        print(
            "IrrigationNeeded + CropWheat"
        )

        print(
            "Derived: ApplyDripMethod"
        )

    print("\nStep 3:")
```

```
    if "ApplyDripMethod" in self.derived_facts:

        print(
            "ApplyDripMethod + "
            "NOT ApplyDripMethod"
        )

        print(
            "Derived: Empty Clause (□)"
        )

        return True

    return False

def remove_soil_dry(self):

    print("\nREMOVING SoilDry")
    print("=" * 50)

    self.facts.discard("SoilDry")

    self.derived_facts = set(self.facts)

    print("Remaining Facts:")

    for fact in sorted(self.facts):
        print(fact)

    print("\nChecking P1:")

    if "SoilDry" not in self.facts:

        print(
            "SoilDry is unavailable."
        )

        print(
            "IrrigationNeeded cannot be derived."
        )

    print("\nChecking P2:")

    if "IrrigationNeeded" not in self.derived_facts:
```

```
            print(
                "IrrigationNeeded is unavailable."
            )

            print(
                "ApplyDripMethod cannot be derived."
            )

        print("\nChecking P3:")

        print(
            "Absence of ApplyDripMethod "
            "does not imply NOT ApplyDripMethod."
        )

        print(
            "Therefore CropAtRisk cannot "
            "be logically derived."
        )


def main():

    agent = AgriculturalReasoningAgent()

    agent.display_facts(
        "INITIAL FACTS",
        agent.facts
    )

    agent.forward_reasoning()

    agent.display_facts(
        "DERIVED FACTS",
        agent.derived_facts
    )

    result = agent.resolution_refutation()

    print("\nFINAL PROOF RESULT")
    print("=" * 50)

    if result:
        print(
```

```
            "ApplyDripMethod is PROVED."
        )
    else:
        print(
            "ApplyDripMethod cannot be proved."
        )

    agent.remove_soil_dry()


if __name__ == "__main__":
    main()
```

4. OUTPUT

```
FINAL PROOF RESULT
==================================================
ApplyDripMethod is PROVED.
```

5. REPORT

The Agricultural Expert Reasoning System was analyzed using **Conjunctive Normal Form (CNF)** and **Resolution Refutation**. The given rules were converted into CNF by eliminating implications and applying logical equivalences. Using the facts SoilDry and CropWheat, IrrigationNeeded was derived first, followed by ApplyDripMethod.

For Resolution Refutation, the goal ApplyDripMethod was negated and added to the knowledge base. Resolution produced the positive conclusion ApplyDripMethod, which contradicted the negated goal and resulted in the **empty clause (□)**. Therefore, ApplyDripMethod was successfully proved.

When SoilDry was removed, IrrigationNeeded could no longer be derived. Consequently, ApplyDripMethod could not be proved. CropAtRisk also does not logically follow because the absence of ApplyDripMethod is not equivalent to the explicit fact ¬ApplyDripMethod. This demonstrates how removing a fact can break a chain of logical inference.

Case Study 4 – Wumpus World Knowledge Agent

1. PROBLEM STATEMENT

An AI agent is navigating the **Wumpus World** using a propositional knowledge base. The agent perceives **Stench at [1,2]**, **Breeze at [1,1]**, and **Glitter at [2,2]**. The knowledge base contains rules for identifying possible Wumpus and Pit locations, locating Gold, and determining safe cells. The objective is to construct the agent's belief state, apply **Modus Ponens and Forward Chaining**, identify possible hazard locations, and determine whether the path **[1,1] → [1,2] → [2,2]** is safe.

2. SOLUTION

(a) Construct the Belief State and Apply Modus Ponens

Given Percepts

The agent perceives:

Stench at [1,2]
Breeze at [1,1]
Glitter at [2,2]

Knowledge Base Rules

Rule 1:

$\text{Stench}[1,2] \rightarrow \text{WumpusAdjacent}[1,2]$

Rule 2:

$\text{Breeze}[1,1] \rightarrow \text{PitAdjacent}[1,1]$

Rule 3:

$\text{Glitter}[x,y] \rightarrow \text{Gold}[x,y]$

Rule 4:

$\neg \text{Wumpus}[x,y] \wedge \neg \text{Pit}[x,y] \rightarrow \text{Safe}[x,y]$

Modus Ponens – Step 1

Given:

$\text{Stench}[1,2]$

and:

$\text{Stench}[1,2] \rightarrow \text{WumpusAdjacent}[1,2]$

Therefore:

$\text{WumpusAdjacent}[1,2]$

Modus Ponens – Step 2

Given:

$\text{Breeze}[1,1]$

and:

Breeze[1,1]→PitAdjacent[1,1]

Therefore:

PitAdjacent[1,1]

Modus Ponens – Step 3

Given:

Glitter[2,2]

and:

Glitter[x,y]→Gold[x,y]

Substituting:

x=2,y=2

Therefore:

Gold[2,2]
Derived Belief State
Stench[1,2]
Breeze[1,1]
Glitter[2,2]

WumpusAdjacent[1,2]
PitAdjacent[1,1]
Gold[2,2]
Important Observation

The rules provided in the question do **not specify exactly which adjacent cell contains the Wumpus or Pit**. Therefore, we can conclude that a Wumpus/Pit is **adjacent**, but we cannot identify one exact cell from the given rules alone.

3. (b) Forward Chaining

Forward Chaining starts with the percepts and applies the rules whenever their conditions are satisfied.

Step 1
Stench[1,2]
|
v
WumpusAdjacent[1,2]
Step 2
Breeze[1,1]
|
v
PitAdjacent[1,1]
Step 3

Glitter[2,2]
|
v
Gold[2,2]

Step 4 – Safe Cell Rule

The rule is:

¬Wumpus[x,y]∧ ¬Pit[x,y]→Safe[x,y]

To apply this rule, the agent must know both:

¬Wumpus[x,y]

and:

¬Pit[x,y]

However, the given knowledge base does **not provide these negative facts**.

Therefore, no specific cell can be proven safe using Rule 4.

Forward Chaining Result

Percept / Fact	Derived Knowledge
Stench [1,2]	Wumpus is adjacent
Breeze [1,1]	Pit is adjacent
Glitter [2,2]	Gold is at [2,2]
Negative Wumpus fact	Not provided
Negative Pit fact	Not provided
Safe cell	Cannot be conclusively derived

4. (c) Path Safety Analysis

The proposed path is:

[1,1]→[1,2]→[2,2]

Cell [1,1]

The agent perceives:

Breeze[1,1]

According to Rule 2:

Breeze[1,1]→PitAdjacent[1,1]

Therefore, there is a possible Pit in a cell adjacent to [1,1].

So [1,1] provides **hazard information**, but the exact Pit location is not determined.

Cell [1,2]

The agent perceives:

Stench[1,2]

According to Rule 1:

Stench[1,2]→WumpusAdjacent[1,2]

Therefore, a Wumpus may be located in a cell adjacent to [1,2].

Thus, the agent does **not have enough information to prove [1,2] safe**.

Cell [2,2]

The agent perceives:

Glitter[2,2]

Using Rule 3:

Glitter[2,2]→Gold[2,2]

Therefore:

Gold[2,2]

The goal of collecting gold is located at [2,2].

However, the knowledge base does not establish:

¬Wumpus[2,2]

and:

¬Pit[2,2]

Therefore, [2,2] also cannot be formally proven safe from the given rules.

Final Path Decision

The path:

[1,1]→[1,2]→[2,2]

cannot be declared safe with the available knowledge.

The rational AI agent should **not assume the path is safe**, because the knowledge base does not provide enough negative information to satisfy the Safe rule.

Conclusion: The gold is definitely at **[2,2]**, but the complete path cannot be logically proven safe using only the given knowledge base.

5. PYTHON CODE

```
class WumpusKnowledgeAgent:

    def __init__(self):

        # Initial percepts
        self.percepts = {
            "Stench[1,2]",
            "Breeze[1,1]",
            "Glitter[2,2]"
        }

        # Knowledge derived by the agent
        self.knowledge = set(self.percepts)

        self.derived_facts = set()

    def display_percepts(self):

        print("\nINITIAL PERCEPTS")
        print("=" * 50)

        for percept in sorted(self.percepts):
            print(percept)

    def modus_ponens(self):

        print("\nMODUS PONENS")
        print("=" * 50)

        # Rule 1
        if "Stench[1,2]" in self.percepts:

            fact = "WumpusAdjacent[1,2]"

            self.knowledge.add(fact)
            self.derived_facts.add(fact)

            print(
                "Stench[1,2]"
                " -> WumpusAdjacent[1,2]"
            )

        # Rule 2
        if "Breeze[1,1]" in self.percepts:

            fact = "PitAdjacent[1,1]"

            self.knowledge.add(fact)
            self.derived_facts.add(fact)

            print(
                "Breeze[1,1]"
                " -> PitAdjacent[1,1]"
            )
```

```
    # Rule 3
    if "Glitter[2,2]" in self.percepts:

        fact = "Gold[2,2]"

        self.knowledge.add(fact)
        self.derived_facts.add(fact)

        print(
            "Glitter[2,2]"
            " -> Gold[2,2]"
        )

def forward_chaining(self):

    print("\nFORWARD CHAINING")
    print("=" * 50)

    print("\nStarting Facts:")

    for fact in sorted(self.percepts):
        print(fact)

    self.modus_ponens()

    print("\nDerived Facts:")

    for fact in sorted(self.derived_facts):
        print(fact)

def check_safe_cell(self, cell):

    wumpus_fact = "NOT Wumpus[" + cell + "]"
    pit_fact = "NOT Pit[" + cell + "]"

    print(
        "\nChecking Safety of Cell",
        "[" + cell + "]"
    )

    print(
        "Required:",
        wumpus_fact,
        "AND",
        pit_fact
    )

    if (
        wumpus_fact in self.knowledge
        and
        pit_fact in self.knowledge
    ):

        print("Result: SAFE")
```

```
        return True

    print(
        "Result: NOT PROVEN SAFE"
    )

    return False

def analyze_path(self):

    print("\nPATH ANALYSIS")
    print("=" * 50)

    path = [
        "1,1",
        "1,2",
        "2,2"
    ]

    print(
        "\nProposed Path:"
    )

    print(
        " -> ".join(
            "[" + cell + "]"
            for cell in path
        )
    )

    safe = True

    for cell in path:

        if not self.check_safe_cell(cell):
            safe = False

    return safe

def display_final_result(self, safe):

    print("\nFINAL RESULT")
    print("=" * 50)

    if "Gold[2,2]" in self.knowledge:

        print(
            "Gold location: [2,2]"
        )

    print(
        "Wumpus information:",
        "Adjacent to [1,2]"
    )
```

```
        print(
            "Pit information:",
            "Adjacent to [1,1]"
        )

        if safe:

            print(
                "Path is logically proven SAFE."
            )

        else:

            print(
                "Path cannot be proven SAFE "
                "with the given knowledge."
            )


def main():

    agent = WumpusKnowledgeAgent()

    agent.display_percepts()

    agent.forward_chaining()

    safe = agent.analyze_path()

    agent.display_final_result(safe)


if __name__ == "__main__":
    main()
```

6. OUTPUT

```
FINAL RESULT
============================================================
Gold location: [2,2]
Wumpus information: Adjacent to [1,2]
Pit information: Adjacent to [1,1]
Path cannot be proven SAFE with the given knowledge.

Process finished with exit code 0
```

7. REPORT

The Wumpus World Knowledge Agent was analyzed using **Modus Ponens and Forward Chaining**. From the given percepts, the agent derived that a Wumpus is adjacent to [1,2], a Pit is adjacent to [1,1], and Gold is located at [2,2]. The Glitter percept directly confirms the gold location.

For safety reasoning, the given Safe rule requires explicit knowledge that both a Wumpus and a Pit are absent from a cell. Since the assessment does not provide such negative facts, the proposed path **[1,1] → [1,2] → [2,2] cannot be logically proven safe**. The rational conclusion is therefore to treat the path as **not proven safe** rather than assuming that it is safe.